

Guía de Práctica Guiada de Laboratorio

Práctica Integradora (60 min): El Monitoreo de Telemetría para Drones de Entrega

Asignatura: Programación Avanzada y Algoritmos (Temas 1.1 a 1.5)

1. Datos Generales y Encuadre Académico

Duración: 60 minutos	Escenario: Laboratorio de C++
Unidad Académica: Ibero Puebla	Modalidad: Individual / Parejas guiadas
Temas Integrados: 1.1 a 1.5 (Clases, Constructores, Parámetros, Composición y Separación .h/.cpp)	Estándar: C++11 / C++17 / C++20

2. Fundamento Teórico e Integración de Conceptos

Esta práctica integradora consolida los cinco pilares del Tema 1 de la asignatura en una única aplicación de telemetría aeronáutica:

- **1.1 Clases y Objetos:** Representación abstracta del componente Bateria y la entidad de dominio DroneEntrega.
- **1.2 Inicialización con Constructores:** Uso de constructores parametrizados y *listas de inicialización de miembros* (operador :) para garantizar un estado inicial consistente sin basura en memoria.
- **1.3 Paso de Parámetros:** Análisis práctico de efectos secundarios. Transferencia por **referencia constante (const Tipo&)** para auditorías eficientes y sin copias, y por **referencia (Tipo&)** para modificaciones reales del estado físico.
- **1.4 Composición de Clases:** Relación de inclusión fuerte "Tiene un" (*Has-a*), donde el ciclo de vida del subobjeto Bateria está supeditado a la existencia del DroneEntrega.
- **1.5 Separación de Interfaz e Implementación:** Arquitectura profesional dividida en archivo cabecera con guardias de preprocesador (Drone.h), archivo de implementación con operador de ámbito (Drone.cpp) y programa cliente (main.cpp).

3. Código Fuente del Sistema (Arquitectura en 3 Ficheros)

Fichero 1: Drone.h (Declaración de Interfaces y Guardias de Preprocesador)

```

#ifndef __DRONE_H__
#define __DRONE_H__

#include
#include

// =====
// CLASE COMPONENTE: Bateria (1.1 y 1.4)
// =====
class Bateria {
private:
    int capacidadmAh;
    float porcentajeCarga;

public:
    // Constructores (1.2)
    Bateria();
    Bateria(int capacidad, float cargaInicial);

    // Métodos de acceso constantes
    float obtenerPorcentaje() const;
    int obtenerCapacidad() const;

    // Métodos modificadores
    void consumirEnergia(float porcentaje);
};

// =====
// CLASE CONTENEDORA: DroneEntrega (1.1, 1.2 y 1.4 Composición)
// =====
class DroneEntrega {
private:
    std::string idDrone;
    std::string estadoVuelo; // "En Base", "En Misión", "Emergencia"
    Bateria bateria;        // Subobjeto por composición (Tiene un)

public:
    // Constructor compuesto con lista de inicialización (1.2 y 1.4)
    DroneEntrega(std::string id, int capBat, float cargaBat);

    // Métodos de consulta constantes
    std::string obtenerID() const;
    std::string obtenerEstado() const;
    float obtenerNivelBateria() const;

    // Métodos operativos
    void despegar();
    void realizarVuelo(float kilometros);
    void aterrizar();
};

#endif // __DRONE_H__

```

Fichero 2: Drone.cpp (Implementación y Operador de Ámbito ::)

```

#include "Drone.h"

// =====
// IMPLEMENTACIÓN DE LA CLASE Bateria (1.5)
// =====
Bateria::Bateria() : capacidadmAh(3000), porcentajeCarga(100.0f) {}

Bateria::Bateria(int capacidad, float cargaInicial)
    : capacidadmAh(capacidad), porcentajeCarga(cargaInicial) {}

float Bateria::obtenerPorcentaje() const { return porcentajeCarga; }
int Bateria::obtenerCapacidad() const { return capacidadmAh; }

void Bateria::consumirEnergia(float porcentaje) {
    porcentajeCarga -= porcentaje;
    if (porcentajeCarga < 0.0f) porcentajeCarga = 0.0f;
}

// =====
// IMPLEMENTACIÓN DE LA CLASE DroneEntrega (1.2, 1.4 y 1.5)
// =====
// Delegación directa al constructor del subobjeto mediante lista de inicialización
DroneEntrega::DroneEntrega(std::string id, int capBat, float cargaBat)
    : idDrone(id), estadoVuelo("En Base"), bateria(capBat, cargaBat) {}

std::string DroneEntrega::obtenerID() const { return idDrone; }
std::string DroneEntrega::obtenerEstado() const { return estadoVuelo; }
float DroneEntrega::obtenerNivelBateria() const { return bateria.obtenerPorcentaje(); }

void DroneEntrega::despegar() {
    if (bateria.obtenerPorcentaje() >= 20.0f) {
        estadoVuelo = "En Misión";
        bateria.consumirEnergia(5.0f); // Despegue consume 5%
        std::cout << " -> [" << idDrone << "] Despegue exitoso. Estado: En Mision." << std::endl;
    } else {
        estadoVuelo = "Emergencia";
        std::cout << " -> [" << idDrone << "] Bateria insuficiente para despegar!" << std::endl;
    }
}

void DroneEntrega::realizarVuelo(float kilometros) {
    if (estadoVuelo == "En Misión") {
        float consumo = kilometros * 4.0f; // 4% de batería por km
        bateria.consumirEnergia(consumo);
        std::cout << " -> [" << idDrone << "] Recorridos " << kilometros
            << " km. Bateria restante: " << bateria.obtenerPorcentaje() << "%" << std::endl;

        if (bateria.obtenerPorcentaje() <= 10.0f) {
            estadoVuelo = "Emergencia";
            std::cout << " -> [" << idDrone << "] ALERTA: Nivel critico de energia!" << std::endl;
        }
    }
}

void DroneEntrega::aterrizar() {
    if (estadoVuelo != "En Base") {
        estadoVuelo = "En Base";
        std::cout << " -> [" << idDrone << "] Aterrizaje completado de forma segura." << std::endl;
    }
}

```

Fichero 3: main.cpp (Programa Cliente y Paso de Parámetros)

```

#include
#include "Drone.h"

// =====
// FUNCIONES EXTERNAS PARA EVALUAR PASO DE PARÁMETROS (1.3)
// =====

// 1. Paso por Referencia Constante (const Tipo&):
// Alta eficiencia (sin copias del dron) + Seguridad (prohibido modificar)
void auditarTelemetria(const DroneEntrega& drone) {
    std::cout << "\n--- [AUDITORIA EN TIEMPO REAL] ---" << std::endl;
    std::cout << " Dron ID: " << drone.obtenerID() << std::endl;
    std::cout << " Estado Actual: " << drone.obtenerEstado() << std::endl;
    std::cout << " Nivel Bateria: " << drone.obtenerNivelBateria() << "%" << std::endl;

    // TRAMPA PEDAGÓGICA (Si descomentas, el compilador bloquea por ser const):
    // drone.despegar(); // Error: no se pueden llamar métodos modificadores
}

// 2. Paso por Referencia (Tipo&):
// Modifica directamente el objeto original sin invocar constructor de copia
void ejecutarMisionEntrega(DroneEntrega& drone, float distanciaKm) {
    std::cout << "\n=== INICIANDO MISION DE ENTREGA DE PAQUETE ===" << std::endl;
    drone.despegar();
    drone.realizarVuelo(distanciaKm);

    drone.aterrizar();
}

// 3. Paso por Valor (Tipo): Demostración de aislamiento e ineficiencia
void simularMisionIlegal(DroneEntrega copiaDrone) {
    std::cout << "\n... Simulando mision en COPIA temporal ..." << std::endl;
    copiaDrone.despegar();
    copiaDrone.realizarVuelo(15.0f);
    std::cout << " Bateria en copia temporal: " << copiaDrone.obtenerNivelBateria() << "%" << std::e
    ndl;
}

int main() {
    std::cout << "===== " << std::endl;
    std::cout << " SISTEMA DE TELEMETRIA Y CONTROL DE DRONES " << std::endl;
    std::cout << "===== " << std::endl;

    // Instanciación con constructor parametrizado (1.1 y 1.2)
    DroneEntrega droneAlpha("DRONE-ALPHA-01", 5000, 100.0f);

    // Auditoría inicial (Paso por referencia constante)
    auditarTelemetria(droneAlpha);

    // Ejecución de misión real (Paso por referencia - Modifica original)
    ejecutarMisionEntrega(droneAlpha, 8.5f);

```

```

// Auditoría post-misión (Comprueba cambios persistentes)
auditarTelemetria(droneAlpha);

// Demostración de paso por valor (No afecta al original)
simularMisionIlegal(droneAlpha);

std::cout << "\nVerificacion en main(): Bateria real tras simulacion = "
           << droneAlpha.obtenerNivelBateria() << "%" << std::endl;

return 0;
}

```

4. Planificación Minuto a Minuto de la Sesión (60 Minutos)

Tiempo	Actividad Docente / Estudiante	Concepto Integrado
00' - 10'	Encuadre del caso de estudio y explicación del mapa de memoria de la composición.	1.1 y 1.4 Composición
10' - 30'	Creación del archivo cabecera Drone.h con guardias de preprocesador y prototipos.	1.5 Separación & Guardias
30' - 45'	Implementación en Drone.cpp de la lista de inicialización del subobjeto Bateria.	1.2 Lista de Inicialización
45' - 55'	Codificación de main.cpp y experimentación del paso de parámetros (& y const &).	1.3 Paso de Parámetros
55' - 60'	Verificación de la compilación modular y resolución del cuestionario de control.	Evaluación y Cierre

5. Cuestionario de Control de Aprendizaje

Pregunta 1: ¿Por qué en la función `auditarTelemetria(const DroneEntrega& drone)` es obligatorio usar el calificador `const` en la firma del parámetro y qué pasaría si intentas llamar al método `despegar()` dentro de ella?

Respuesta esperada: El calificador `const` asegura que el objeto no sea modificado. El compilador bloqueará cualquier intento de invocar `despegar()` porque este método no está declarado como de solo lectura (`const`) y altera el estado del dron.

Pregunta 2: En la clase `DroneEntrega`, ¿cuál es el beneficio técnico de inicializar el subobjeto `bateria` en la lista de inicialización de miembros (`: bateria(capBat, cargaBat)`) en lugar de hacer una asignación dentro del cuerpo del constructor?

Respuesta esperada: Evita la doble inicialización en memoria. Si se usara asignación en el cuerpo, C++ primero invocaría implícitamente el constructor por defecto de `Bateria` y luego sobrescribiría los datos, gastando ciclos de reloj innecesarios.

Pregunta 3: ¿Qué función cumplen las directivas `#ifndef __DRONE_H__`, `#define __DRONE_H__` y `#endif` en el archivo `Drone.h`?

Respuesta esperada: Actúan como guardias de preprocesador. Garantizan que el compilador procese la declaración de las clases una sola vez, evitando errores de redefinición si la cabecera es incluida múltiples veces en el proyecto.

6. Rúbrica de Evaluación Académica (Ibero Puebla)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (5)
Integración POO (30%)	Aplica clases, composición y constructores con sintaxis perfecta.	Aplica composición pero omite la lista de inicialización de miembros.	Implementa las clases de forma monolítica sin relación limpia.	No logra definir la composición o comete errores graves.
Separación .h/.cpp (25%)	Divide correctamente los 3 ficheros con guardias e inclusión limpia.	Divide en ficheros pero omite las guardias de preprocesador.	Presenta fallas de enlazado o dobles inclusiones en compilación.	No realiza la separación física de archivos.
Paso de Parámetros (25%)	Demuestra el uso correcto de const& y & diferenciando sus efectos.	Utiliza referencias pero no aplica el modificador const en auditorías.	Usa únicamente paso por valor provocando copias ineficientes.	Desconoce la sintaxis de referencias en C++.
Cuestionario y Reporte (20%)	Responde las 3 preguntas con sustento técnico y entrega a tiempo.	Responde de forma clara pero omite detalles teóricos de memoria.	Responde incompletas las preguntas de control.	No entrega el cuestionario de evaluación.

7. Referencias Bibliográficas (APA 7ª ed.)

- Ceballos Sierra, F. J. (2015). *Programación orientada a objetos con C++* (4.ª ed.). RA-MA Editorial.
- Finochietto, J. M. (2010). *Programación Orientada a Objetos en C++*. Universidad Nacional de Córdoba (UNC-CONICET).
- García Peñalvo, F. J., & Hernández Simón, J. A. (2005). *2 - Clases y objetos en C++*. Departamento de Informática y Automática, Universidad de Salamanca.
- Garrido Carrillo, A. (2016). *Prácticas con C++: metodología de la programación* (2.ª ed.). Editorial Universidad de Granada.
- González Pérez, A. (2025). *El gran libro de programación en C++* (1.ª ed.). Marcombo.
- Google (2026). *Gemini Notebook [Asistente de Inteligencia Artificial]*. <https://notebook.google.com/>